Agent Postmortem Generator

Part 13

Performance & Scalability Strategy

One-line purpose

Define the performance envelope, scalability boundaries, and optimization rules that keep the MVP fast, deterministic, and maintainable on a local developer machine.

1. Executive Overview

This module defines how the project should remain fast enough for daily local use while staying simple enough for a small team to maintain. It does not add new runtime behavior. It sets performance budgets, identifies the main scaling pressure points, and establishes the rules for when optimization is allowed.

The module exists because the tool must analyze traces, reconstruct failure chains, and generate reports without feeling sluggish on a laptop. If the pipeline becomes slow, memory-heavy, or unpredictable, engineers will stop trusting it and the product loses its value. Performance in this project is not a bonus feature; it is part of the trust contract.

In the system architecture, this module spans the entire pipeline as a set of non-functional constraints. It influences parser design, trace model shape, analysis execution, graph generation, report rendering, CLI behavior, and export decisions. It does not own any business logic. It owns the rules that keep the rest of the system efficient.

```
Input -> Parser -> Validation -> Normalizer -> Canonical Trace -> Analysis -> Postmortem -> Output
          |            |             |                 |                 |          |
      bounded I/O   one-pass     early reject     immutable data     pure ops     memoize only   stream when possible
```

2. Problem Statement

The exact problem is not raw compute speed. The problem is uncontrolled cost growth: repeated traversals, duplicate allocations, full-file loads when streaming would suffice, expensive recomputation, and output generation that scales worse than the input.

This is hard because the project is inherently trace-heavy. Execution traces can contain many events, nested payloads, tool outputs, and repeated metadata. The same trace may need to be parsed, normalized, analyzed, visualized, and rendered into multiple outputs. Without strict rules, each layer would reinvent the same work and the local-first UX would collapse under its own weight.

The MVP needs this module because even a narrow product can still become unusable if it is careless about memory and repeated work. The performance strategy makes the architecture explicit about what is allowed to grow linearly, what must stay bounded, and what must never be done twice.

3. Goals

Goal	What it means in practice
Fast enough for local use	Typical developer-sized traces should process quickly enough that the tool feels interactive rather than batch-only.
Predictable resource use	Memory and CPU consumption should scale primarily with input size, not with accidental duplication of work.
Deterministic performance behavior	The same input should produce similar runtime characteristics across runs on the same machine, aside from normal background noise.
Bounded complexity	No optimization should introduce hidden global state, cross-module coupling, or hard-to-debug caching behavior.
Simple to maintain	Performance rules must be understandable by 2 to 3 engineers without a dedicated infrastructure team.

This module must achieve three things: establish clear performance budgets, provide a decision framework for optimization, and keep future scale work isolated from the core MVP path.

4. Non-Goals

- No distributed processing.
- No remote compute, background worker fleet, or queueing system.
- No speculative precomputation for unknown future workloads.
- No aggressive compression, indexing, or storage-engine design in the MVP.
- No per-framework tuning beyond adapter boundary constraints.
- No premature parallelization of a single trace unless it can be proven useful and safe.
- No probabilistic optimization that changes results or ordering.

Later versions may introduce batch comparison, cached run artifacts, incremental indexing, or multi-trace throughput optimization. Those are explicitly deferred because they are not needed to prove the core debugging loop.

5. Responsibilities

This module owns performance policy, not implementation details of business logic.

- Define acceptable latency, throughput, and memory posture for each major pipeline stage.
- Identify likely bottlenecks and preferred mitigation patterns.
- Specify what may be cached, memoized, streamed, or parallelized.
- Document the conditions under which optimization is allowed.
- Preserve determinism and reproducibility while improving speed.

This module must never own trace semantics, incident interpretation, report content, adapter specifics, or UI behavior. If it starts deciding what the failure means, it has already failed its job.

6. Inputs

Inputs to this strategy are the architectural constraints and the workload shapes produced by the MVP pipeline.

Input type	Examples	Why it matters	Handling rule
Trace size	event count, payload size, nested tool outputs	Determines parse, normalize, and analysis cost	Treat as the primary scaling driver
Output mix	Markdown, Mermaid, JSON, future PDF	Affects rendering and export cost	Render only what the user requested
Execution mode	CLI run, UI inspection, test execution	Changes latency expectations	Keep the runtime path consistent across modes
Environment	laptop CPU, memory, filesystem speed	Affects what is practical locally	Assume modest hardware and avoid reliance on high-end machines
Module boundaries	canonical trace model, pure analysis, adapters	Determines what can be optimized safely	Optimize only where ownership is clear

Assumptions: traces are local files; the MVP usually processes one trace at a time; report generation is a post-analysis step; and the canonical trace model is the shared contract that all downstream modules can rely on.

7. Outputs

- A documented performance envelope for the project.
- A list of preferred optimization patterns and forbidden anti-patterns.
- Module-level budget guidance for time, memory, and I/O.
- Operational recommendations for profiling and regression detection.
- A clear escalation path for future scale work.

Human-readable outputs matter here: engineering decisions are the product. This module should produce written guidance that future module owners can apply without interpretation theater.

8. Internal Design

The internal design is intentionally simple. It has four parts:

- Budget definition: establish what 'fast enough' means for the MVP.
- Bottleneck classification: identify which stages are likely to dominate runtime or memory.
- Optimization policy: define how to improve performance without changing meaning or order.
- Escalation policy: define when a problem is worth solving now versus later.

Workflow:

```
1. Ingest architectural context and expected trace shape
2. Assign the dominant cost to each pipeline stage
3. Classify costs as unavoidable, optimizable, or deferred
4. Set budgets and guardrails
5. Require profiling before any non-trivial optimization
6. Re-evaluate budgets when a new module changes workload shape
```

Important decision: the strategy should optimize the common case first and reject engineering work that only helps extreme inputs unless there is evidence that those inputs will be routine.

9. Data Model

Entity	Purpose	Key fields	Why it exists
Performance budget	Numeric and qualitative guardrails	stage, target latency, memory ceiling, notes	Prevents vague goals like 'make it faster'
Bottleneck class	Standardized cost bucket	stage, symptom, mitigation, priority	Helps the team reason about where time goes
Optimization rule	Approved performance behavior	allowed, forbidden, conditions, owner	Keeps changes deterministic and reviewable
Scale trigger	Condition that warrants revisiting design	threshold, signal, next action	Avoids premature redesign while still detecting growth

Relationships are simple: one budget may map to many bottleneck classes, one bottleneck class may have many optimization rules, and scale triggers point to either a rule change or a future architecture review.

10. Algorithms / Processing Logic

This module does not run in the product path, so its 'algorithm' is a decision process for engineering work.

- Measure the baseline of the current MVP pipeline on representative local traces.
- Find the dominant stage by time, allocation, or output size.
- Classify the cost as unavoidable input work, avoidable repetition, or avoidable rendering overhead.
- Prefer structural fixes before micro-optimizations.
- Only permit caching if the cached artifact is pure, deterministic, bounded, and easy to invalidate.
- Only permit parallelism if it does not change output order or degrade reproducibility.
- Reject optimization that increases coupling or makes the core logic harder to test.

Edge cases include very large traces, huge tool outputs, and repeated analyses of the same file. The default rule is still restraint: if the workload is outside MVP intent, document the limitation instead of building a premature platform.

11. Tradeoffs

Tradeoff	Chosen side	Reason

Simplicity vs flexibility	Simplicity	A small team can maintain simple rules far better than a generic performance framework.
Generality vs specificity	Specificity	The product has one core workflow; broad abstractions would mostly add confusion.
Determinism vs heuristic inference	Determinism	The tool is trusted because it behaves predictably, not because it guesses well.
Optimization now vs later	Later unless proven	The MVP should solve visible pain, not hypothetical scale problems.

The preferred design is conservative. It favors bounded local work, explicit measurement, and module-specific budgets over shared magic. That is less glamorous and more correct.

12. Risks and Failure Modes

- Repeated full-trace scans can quietly multiply cost.
- Overly rich canonical data can create memory pressure.
- Graph generation or rendering may dominate runtime if built without size awareness.
- Caching can become stale or non-deterministic if invalidation is fuzzy.
- Premature parallelism can make debugging harder and outputs less stable.
- Future adapters may introduce bigger payloads than the MVP expects.

The hidden complexity is not raw CPU usage; it is cost accumulation across layers. Each layer may look cheap in isolation and still produce an expensive system when combined. The strategy must therefore be cross-cutting, not local-only.

13. Testing Strategy

- Unit tests for budget classification and optimization-rule evaluation.
- Fixture-driven tests using small, medium, and intentionally large trace samples.
- Integration tests that measure whether the pipeline remains within expected bounds on representative local hardware.
- Regression tests that compare current runtime and memory to stored baselines.
- Stress tests that validate graceful degradation rather than crashes.

Performance tests should not be flaky. They should use fixed fixtures, controlled hardware assumptions where possible, and wide enough thresholds to account for normal machine noise. The goal is signal, not ritual sacrifice to a benchmark dashboard.

14. Implementation Checklist

- Define the MVP workload profile and what counts as 'typical' input.

- Set stage-level budgets for parsing, normalization, analysis, generation, and export.
- Document the allowed optimization techniques for each stage.
- Add profiling fixtures before optimizing anything.
- Create a regression baseline for the main local execution path.
- Add a review gate requiring profiling evidence for any change that claims to improve speed.
- Document the fallback position for workloads that exceed MVP scale.

15. Future Expansion

- Batch processing of multiple traces with controlled concurrency.
- Cached run artifacts for repeated analysis of the same trace.
- Optional precomputed indexes for very large traces.
- Parallel export of independent output formats.
- Better handling of extremely large adapter payloads.
- A lightweight benchmark suite for release gating.

These expansions naturally follow from the MVP because they improve throughput without changing the meaning of the system. They should still respect the same deterministic and local-first constraints.

16. Final Recommendation

This design is MVP-ready if it remains a strategy document and not a stealth architecture rewrite. The best simplification is to keep the first version brutally honest: optimize the common local path, document the limits, and defer all scale fantasies until there is evidence that users need them.

The module should be adopted as a standing rule set for the project: measure first, optimize second, preserve determinism always.